

Jimma University

Faculty of Computing and Informatics

Computer Science Department

Web Programming

Jimma, Ethiopia.

Introduction to JavaScript

- JavaScript is a high-level programming language that adds interactivity to your website. This happens in the behavior of responses when buttons are pressed or with data entry on forms; with dynamic styling, with animation, etc.
  - It is designed to add interactivity to HTML pages.
  - It is a scripting language (**a lightweight programming language**)
  - Usually embedded directly into HTML pages
  - JavaScript gives HTML designers a programming tool
- Note that JavaScript code did not need to be compiled
  - JavaScript is an interpreted language
  - A portion of the browser software that reads and executes JavaScript is an interpreter

Cont...

❖ What can JavaScript Do?

- ❖ JavaScript can put dynamic text into an HTML page
- ❖ Running code in response to certain events occurring on a web page.
- ❖ JavaScript can read and write HTML elements
- ❖ JavaScript can be used to validate data
- ❖ Store useful values inside variables.
- ❖ JavaScript can be used to detect the visitor's browser
- ❖ JavaScript can be used to create cookies
- ❖ Store and retrieve information on the visitor's computer

JavaScript can be run within a variety of environments:

- Web browsers
- Web servers
- Application containers (general-purpose programming)

Running the JavaScript

- Any time you include JavaScript in an HTML document, you must enclose it inside a tag

  <script >...</script>

  - These tags alert the browser program to begin interpreting all the text between these tags as a script.
  - Other scripting languages such as VBScript also take advantage of these script tags

- Hence, you must specify the precise name of the language in which the enclosed code is written to **JavaScript**.

  - When the browser receives this signal, it employs its **built-in JavaScript interpreter to handle the code**. (JavaScript interpreter: V8, Chakra, Spider monkey, and JavaScript core webkit)

```
<script language="JavaScript" or lang="JavaScript">
        //your script here
</script>
```

Cont...

- Here are some tips to remember when writing JavaScript commands:
  - JavaScript code is case-sensitive (e.g. age and Age are different variables)
  - White space between words and tabs are ignored
  - Line breaks are ignored except within a statement
  - JavaScript statements end with a semi-colon (;)

- **There are three ways to add JavaScript commands to your Web Pages.**
  - ***Embedding code***
  - ***Inline code***
  - ***External file***

Adding JavaScript

I. External File

- If you want to run the **same JavaScript on several pages**, you can write the JavaScript in an external file. Save the external JavaScript file with a **.js file extension**.
- The external script cannot contain the <script></script> tags!
- You can use the SRC attribute of the **<SCRIPT> tag** to call JavaScript code from an external text file.
- It is called by the following tag:

  <script lang="text/JavaScript" src = *"filename"*> </script>
- The script tag should be placed in the HTML page's head

II. Embedding code: Scripts in <head>

- Scripts to be executed when they are called, or when an event is triggered, are placed in functions.
  - Put your functions in the head section; this way, they are all in one place, and they do not interfere with page content.

```
<head>
<script lang="javascript">
    function message(){
       alert("This alert box was called with the onload
       event");
    }
</script>
</head>
```

III. Inline code: Scripts in <body>

- If you don't want your script to be placed inside a function, or if your script should write page content, it should be placed in the body section.

```
<html>
<head>  </head>
<body>
    <script lang="javascript">
        document.write("This message is written by JavaScript");
    </script>
</body>
</html>
```

Input-Output in JavaScript

- In JavaScript, input-output can be done in different ways:

```
document.write("message to display");
alert("message to display");
prompt("message to display", "default value");
confirm("message to display");
console.log("Hello world")
```

- **documents.write** method writes a string to the web page.
  - Anything between double quotes will be displayed as it is on the web page.
  - However, if there is something out of quotes, it is **evaluated as an expression,** and the result will be sent to the web page.

Cont...

- The **alert method** produces a browser alert box.
- These are useful for **debugging and learning the language**. However, they are not a good way to communicate with the users.
- alert() displays a modal window that presents a message to the user with a single OK button to dismiss the dialog box.



- The **prompt display** includes a **message**, a **field for user input**, and **two buttons (OK and Cancel).**
  - The prompt("") returns a string of text entered by the user.
  - It takes two parameters:
    - a message providing the prompt for the response, and
    - a default string that is used to fill in the text field.



Cont...

- A **confirm dialog box** presents a message in a modal dialog box along with OK and Cancel buttons.
  - Such dialog boxes can be used to ask the user a question, **usually before performing undoable actions.**
  - The dialog box returns a Boolean value of *Ok=true;* and *Cancel=false*;
- Example:

```
var adult = confirm("Are you sure you are older than 18 years?")
if(adult)
   alert("Yes");
else
   alert("No");
```

Working with Variables and Data

- In JavaScript, the value of a variable can be one of several types. The type of a variable is dynamic: it depends on the type of data it contains.
- The table lists JavaScript's formal data types, with examples of the values.

Type	Example	Description
String	"John"	a series of characters inside quotation marks
Number	4.5	any number not inside quotes
Boolean	True	a logical true or false
Null	Null	completely devoid of any value. Represents the intentional absence of any object value.
Object		Class that is defined by its properties and methods
Undefined		Undefined (the value of a newly created variable). **A variable has been declared but has not been assigned a value.**

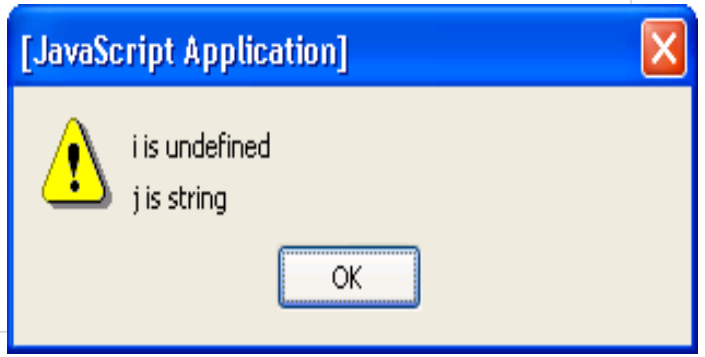
Cont...

- To declare a variable, we use the **var, let, and const keywords**, followed by the name of the variable.
- Therefore, to declare a new variable called **myAge**:

  var myAge;

- Apply variable declaration rules: must begin with a letter or underscore (_), must contain only letters, underscores, and digits (or certain other characters), and not be a reserved word
  - It is possible to assign a value using an **equal (=) sign**. JavaScript is loosely typed.
- **Variables can store any type of data**. **var value="Hello world!"; value = 30.5;**
- **The `typeof`** operator returns a string related to the data type
- Syntax: **`typeof`** *expression*

```
// TypeOf.js
var i;
var j;
j = "Not a number";
alert("i is " + (typeof i) + "\n" +
      "j is " + (typeof j));
```



Values returned by typeof for various operands

Operand Value	String typeof Returns
null	"object"
Boolean	"boolean"
Number	"number"
String	"string"
native Object representing function	"function"
native Object not representing function	"object"
declared variable with no value	"undefined"
undeclared variable	"undefined"
nonexistent property of an Object	"undefined"

- **A variable will automatically be created if a value is assigned to an undeclared identifier:**

```
testing = "Does this work?";
window.alert(testing);
```

Comments

- JavaScript supports two types of comments:
  - Comments on a single line are preceded by //.
  - Comments that span multiple lines are preceded by /* and followed by */
- **Example:** the following example shows two comments:

```
//This next line prints text into the document
document.write("This line came from some JavaScript");
/* This is a multiple-line comment. This line shows an
   alert so we know things worked  properly */
alert("The text has been printed");
```

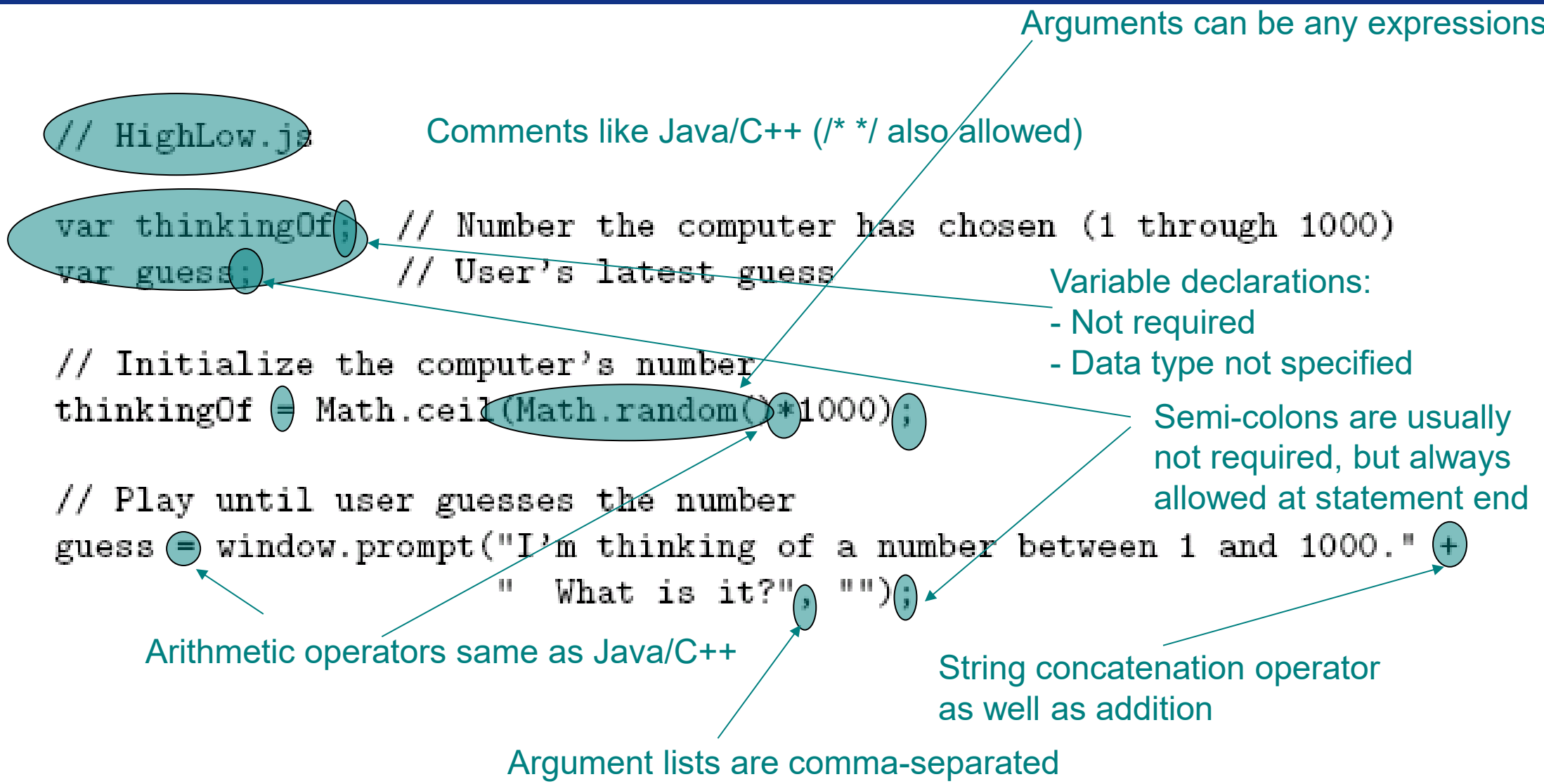
Operators and Expressions

- An operator performs some kind of calculation (operation) or comparison with two values to reach a third value.
- Generally, operators can be broadly categorized into four:
  - **Arithmetic operators,**
  - **Assignment operators**
  - **Comparison operators and**
  - **Logical operators.**

- Operators are used to create compound expressions from simpler expressions
- Operators can be classified according to the number of operands involved:
  - Unary: one operand (*e.g.*, typeof i)
    - Prefix or postfix (*e.g.*, ++i or i++)
  - Binary: two operands (*e.g.*, x + y)
  - Ternary: three operands (conditional operator)

```
(debugLevel>2 ? details : "")
```

Cont...


Arguments can be any expressions
// HighLow.js
Comments like Java/C++ (/* */ also allowed)
var thinkingOf; // Number the computer has chosen (1 through 1000)
var guess; // User's latest guess
Variable declarations:
- Not required
- Data type not specified
// Initialize the computer's number
thinkingOf = Math.ceil(Math.random()*1000);
Semi-colons are usually not required, but always allowed at statement end
// Play until user guesses the number
guess = window.prompt("I'm thinking of a number between 1 and 1000." +
" What is it?", "");
Arithmetic operators same as Java/C++
String concatenation operator as well as addition
Argument lists are comma-separated

Data Type Conversions

- The type of data in an expression can trip up some script operations if the expected components of the operation are not of the right data type.
  - **JavaScript tries its best to perform internal conversions to head off such problems.**
- In a simple arithmetic statement that adds two numbers together, you get the expected result:

```
3 + 3; // result = 6

3 + "3" // result = "33"

3 + 3 + "3" // result = "63"
```

- JavaScript provides two built-in functions to convert string representations of numbers to true numbers:
  - parseInt(string [,radix])
  - parseFloat(string [,radix])
    - radix, which is optional, specifies the base of the number to convert to: hexadecimal, octal, or decimal.
  - parseInt("42") // result = 42
  - parseInt("42.33") // result = 42

Automatic Type Conversion

- **Binary operators +, -, *, /, %** convert both operands to Number
  - Exception: If one of the operands of + is String, then the other is converted to a String
- **Relational operators <, >, <=, >=** convert both operands to Number
  - Exception: If both operands are String, no conversion is performed and lexicographic string comparison is performed
- **Operators ==, !=** convert both operands to Number
  - Exception: If both operands are String, no conversion is performed (lex. comparison)
  - Exception: values of **Undefined and Null** are equal
  - Exception: two Objects are equal only if they are references to the same object
- **Operators ===, !==** are strict:
  - Two operands are === only if they are of the same type and have the same value
- **Unary +, -** convert their operand to Number
- **Logical &&, ||, !** convert their operands to Boolean

Working with Conditional Statements

- Conditional statements are used to perform different actions based on conditions.
- Broadly, there are two ways to execute code conditionally:
  - If statement
  - switch statement

Example:

```
if (myAge < 18) {
        alert("Sorry, you cannot vote.")
}
```

```
switch (fruittype) {
  case "Apples":
      document.write("Apples are $0.32 a pound.<br>");
      break;
  case "Bananas":
      document.write("Bananas are $0.48 a pound.<br>");
      break;
  case "Mangoes":
  case "Papayas":
      document.write("Mangoes and papayas are $2.79 a
pound.<br>");
      break;
  default:
      document.write("Sorry, we are out of " + fruittype + ".<br>");
}
```

Working with Loops

- A loop is a set of commands that executes repeatedly until a specified condition is met.
- JavaScript supports the for, do while, and while loop statements.
  - In addition, you can use the *break* and *continue* statements within loop statements.
- Another statement, for...in, executes statements repeatedly but is used for object manipulation.
- There are three loop statements in JavaScript:
  - for Statement
  - do...while Statement
  - while Statement

Cont...

A for loop repeats until a specified condition evaluates to false.

```
var counter = 10;
var sum = 0;
 for (var i = 0; i <= counter; i++) {
          sum = sum + i;
 }
document.write("the sum is " + sum);
```

A *do...while statement* looks as follows:

```
var i=1;
do{
    document.write(i);
    i += 1;
} while (i < 5);
```

A while statement looks as follows:

```
while (condition)
{
        statement
}
```

Functions in JavaScript

- Functions are one of the fundamental building blocks in JavaScript.
- A function is a set of statements that performs a specific task.
- To use a function, you must first define it, then your script can call it.

Defining Functions

Declaration always begins with keyword `function`, no return type

- A function definition consists of a function keyword, followed by:
  - The name of the function.
  - A list of arguments to the function, separated by commas.
  - The statements that define the function, enclosed in curly braces { }.
- The syntax to define function:

Identifier representing function's *name*

```
function functionName( [parameter1]...[,parameterN] ) {
    statement[s]
}
```

Formal parameter list

One or more statements representing *function body*

Cont...

- Function names have the same restrictions as variable names.
  - It is possible to use multiword names with the interCap format that start with a verb because functions are action items.
- **Example:** the following code defines a simple function named square:

```
function square(number) {
        return number * number;
}
```

- Arrow function useful for writing shorter function expressions

```
const functionName = (parameters) => {
   // function body
};
```

```
const add = (a, b) => {
    return a + b;
};
document.write(add(2, 3)); // Output: 5
```

Cont...

- Function expression

```
const  mult= function(a,b){
return a*b;
}
```

```
( () => {
    console.log("Arrow IIFE!");
    }) ();
```

- **Anonymous function: a** function without a name, often used as an argument in other functions.
  - **(function(){**

    console.log("Runs immediately!");

    }) ();

```
(function(name){
    console.log("Hello "+ name);
    }) ("Abebe");
```

  - Called immediately invoked function expressions. Instead of waiting to be called later, the function executes right away.
  - It is useful for setup tasks that should run once

Cont...

- JavaScript checks neither the type nor the number of parameters in a function call
  - Formal parameters have no type specified
  - Extra actual parameters are ignored
    - If there are fewer actual parameters than formal parameters, the extra formal parameters remain undefined
- This flexibility is typical of many scripting languages; different numbers of parameters may be appropriate for different uses of the function.

```
function greet(name) {
    document.write('Hello, ' + name + '!');
}
greet(); // Output: Hello, undefined!
```

```
const mult  =  function add(a, b) {
      return a + b;
}
document.write(add(5, 10, 15)); // Output: 15
```

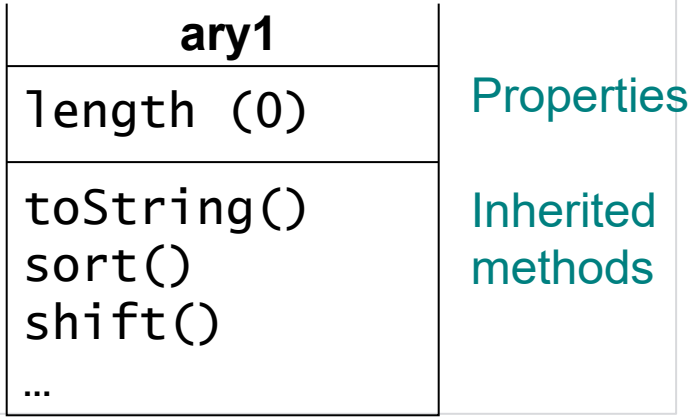
Arrays

- The Array built-in object can be used to construct objects with special properties that inherit various methods
- An array is an ordered collection of data. An array in JavaScript is heterogeneous and can store numbers, strings, objects, or even other arrays.
- To initialize an array for your script, use the new keyword to construct the object for you while assigning the array object to a variable of your choice:
- var myArray = new Array(n); where n is the number of entries you anticipate for the array.

Example: an array that stores the names of planets

```
var planet = new Array(3);   //an array with 3 entries
planet[0] = "Mercury"
planet[1] = "Venus"
planet[2] = "Earth"
```

```
var ary1 = new Array();
```



Cont...

- You can also create array by directly giving the values:
  - var planet = new Array("Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune", "Pluto");
- You can also create the planets array like:
  - var planet = ["Mercury", "Venus", "Earth", "Mars", "Jupiter", "Saturn", "Uranus", "Neptune", "Pluto"];
- In general, you can create an array in three different ways :

```
arrayName = new Array(arrayLength);

arrayName = new Array(element0, element1, ..., elementN);

arrayName = [element0, element1, ..., elementN];
```

Cont...

Access an Array

- You can use the document.write() method to display all the content of the Array.
- This will produce one long text string that is composed of each element of the array like this:
  - document.write(planet);
- This produces the following output:

  [Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune, Pluto]
- You can refer to a particular element in an array by **using the index number**.
  - The index number starts at 0 and ends at n-1 for an array of n entries.
- For example, to access the fifth planet in the planet's array, we use:
  - document.write(planet[4]); //prints Jupiter

Cont...

- To output each element of the array individually, you can set up a for () statement like this:

```
for (var i=0; i < planet.length; i++) {
        document.write(planet[i] + "<BR>"); }
```

Deleting Array Entries

- You can always set the value of an array entry to null or an empty string to wipe out the data.
  - But with the delete operator, you could not completely remove the element.
- Deleting an array element eliminates the index from the list of accessible index values
  - **But it does not reduce the array's length**

```
planet.length;          // result: 9
delete planet[2];
planet.length;          //result: 9
document.write(planet[2]); //result: undefined
```

Method	Description
filter()	Creates a new array with all of the elements of this array for which the provided filtering function returns true.
indexOf(value)	Returns the first (least) index of an element within the array equal to the specified value, or **-1** if none is found.
join(separator)	Joins all elements of an array into a string using separator
lastIndexOf(value)	Returns the last (greatest) index of an element within the array equal to the specified value, or -1 if none is found.
pop()	Removes the last element from an array and returns that element.
push(value)	Adds one or more elements to the end of an array and returns the new length of the array.
reverse()	Reverses the order of the elements of an array − the first becomes the last, and the last becomes the first.
shift()	Removes the first element from an array and returns that element.
slice(start [,end])	Extracts a section of an array and returns a new array.
splice(start [,deletecount [,item1 [,item2 [,…itemN]]]]))	Replaces and/or removes elements from an array.
toString()	Returns a string representing the array and its elements.
unshift(value)	Adds one or more elements to the front of an array and returns the new length of the array.

Cont...

```
function arrayFunction() {
    var grade = new Array("70", "60", "80",
    "90", "20");
    document.write("<br> Popped: " +
    grade.pop());
    document.write("<br> After poping:");
    for(var i=0;i<grade.length; i++)
        document.write(" " + grade[i]);
    grade.reverse();
    document.write("<br>Reversed: ");
    for(var i=0;i<grade.length; i++)
        document.write(" " + grade[i]);
    document.write("<br>Search 80: " +
    grade.indexOf("80"));
    document.write("<br>Converted to string: "
    + grade.toString());
    var slicedValue = grade.slice(2);
    document.write("<br>Sliced: " +
    slicedValue);
    document.write("<br>Spliced: " +
    grade.splice(1));
    grade.unshift("500","6000");
    document.write("<br>Final: " + grade);
}
```

Cont...

- This produces the following output:

```
//70, 60, 80, 90, 20
Popped: 20
After poping: 70 60 80 90
Reversed: 90 80 60 70
Search 80: 1
Converted to string: 90,80,60,70
Sliced: 60,70
Spliced: 80,60,70
Final: 500,6000,90
```

```
let numbers = [1,2,3,4,5,6];
let even = numbers.filter( num=>num%2===0 )

let fruits = ["Appel", "Banana", "Mango", "Avocado"];
let afruits = fruits.filter(fruit=>fruit.startsWith("A"));
console.log(afruits)

    let students =[
    {name:" Abeba", score:85},
    {name:" Chala", score:45},
    {name:" Kebede", score:95}
    ];
let passed = students.filter(student => student.score
>=50);
console.log(passed)
```

Managing Events

- Events are occurrences generated by the browser, such as loading a document, or by the user, such as moving the mouse, clicking a button, pressing a key; that the browser detects and allows your code to respond to.
- They are the user and browser activities to which we may respond dynamically with a scripting language like JavaScript.

Event	Event Handler	Description
Load	onLoad	Browser finishes loading a Web document
Unload	onUnload	Visitor requests a new document in the browser window
Mouseover	onMouseOver	Visitor moves the mouse over some object in the document window
Mouseout	onMouseOut	Visitor moves the mouse off of some object in the document window
MouseDown	onMouseDown	A mouse button was pressed
MouseMove	onMouseMove	The mouse moved
MouseUp	onMouseUp	The mouse button was released
Select	onSelect	Text has been selected.
Click	onClick	Visitor clicks the mouse button
Focus	onFocus	Visitor gives focus to or makes active a particular window or form element by clicking on it or tabbing to it
Blur	onBlur	A form field lost the focus (user moved to another field)
Change	onChange	Visitor changes the data selected or contained in a form element
Submit	onSubmit	Visitor submits a form
Reset	onReset	Visitor resets a form
Abort	onAbort	An image failed to load
Change	onChange	The contents of a field has changed
DblClick	onDblClick	User double-clicked on this item
Error	onError	An error occurred while loading an image
Keydown	onKeyDown	A key was pressed
KeyPress	onKeyPress	A key was pressed or released
KeyUP	onKeyUp	A key was released

Cont...

- Example: a program that adds or subtracts two numbers when the respective button is clicked

```
<html><head>
<script language="JavaScript">
function adder(num1, num2){
        var sum = 0;
        sum = num1 + num2;
        document.write("The sum is " + sum);
}
function subtractor(num1, num2){
        var difference = 0;
        difference = num1 - num2;
document.write("The difference is " + difference);
}
</script>
</head><body>
<form name="event_example">
<input type="button" value="add" name="add"
onClick="adder(10,30)">
<input type="button" value="subtract"
name="subtract" onClick="subtractor(20,50)">
<a href="" onclick="adder(30,40)"> Add </a>
</form></body></html>
```

JavaScript objects

- JavaScript has a powerful and flexible object model.
  - Unlike the Java programming language, JavaScript is a classless language:
    - The **behavior and state of an object are not defined by a class**, but rather by other objects
  - An object is a dynamic data structure that stores related data and functionality as key—value pairs. It is an object that groups data and behavior together.
- Three ways to create an object
  - **You can use an object literal**
  - **You can use new to create** a "blank" object
  - **You can use a constructor**

Three ways to create an object

- **You can use an object literal:**
  - var course = { number: "CIT597", teacher="Dr. Dave" } course.credits = 4;
- **You can use new to create** a "blank" object, and add fields to it later:

```
var course = new Object();
course.number = "CIT597";
course.teacher = "Dr. Dave";
```

```
document.write("the course number is "+course.number +"the course teacher is "+course.teacher );
```

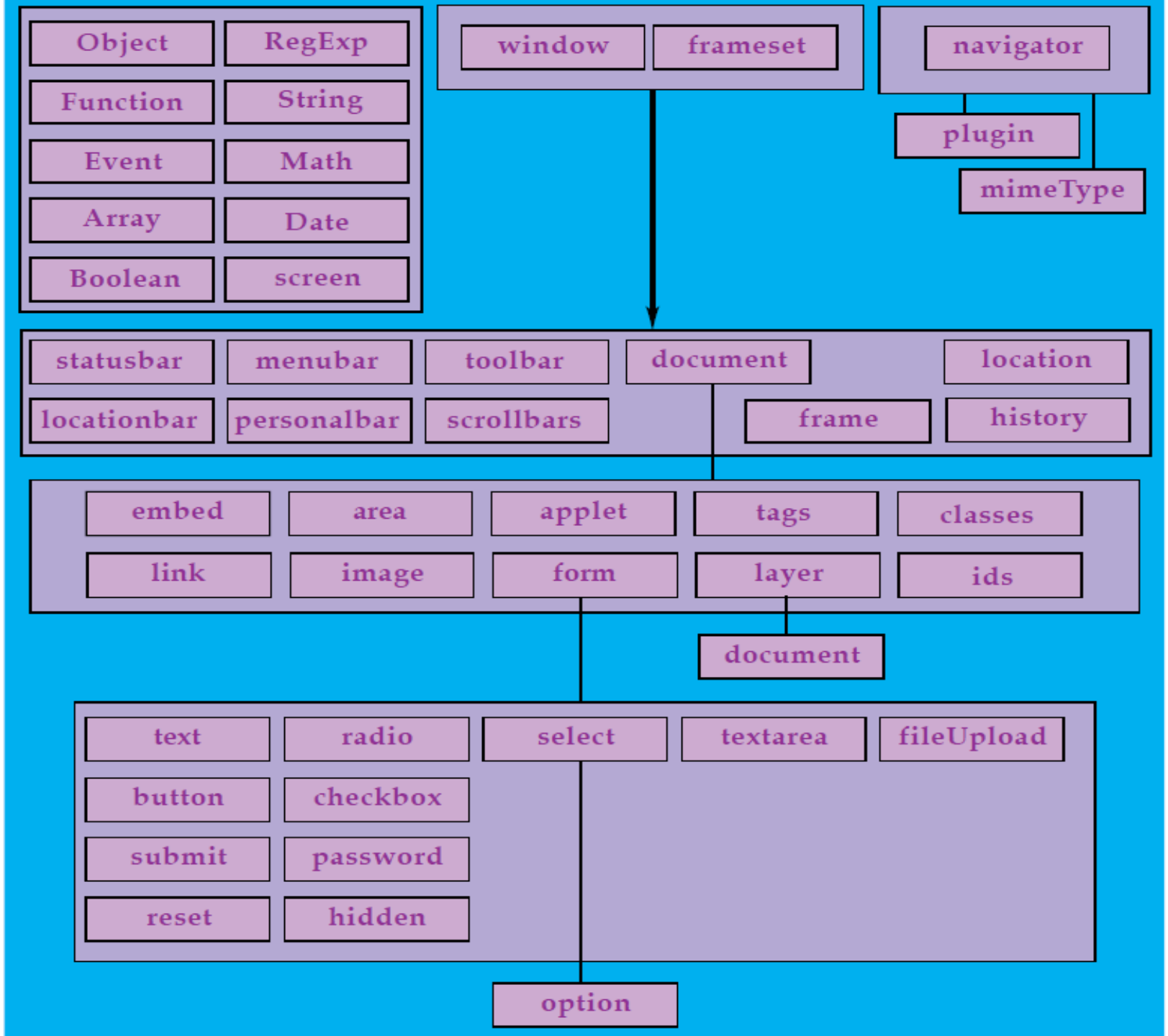
- **You can write and use a constructor:**

```
function Course(n, t) {  // best placed in <head>
    this.number = n;
    this.teacher = t;
}
```

```
var course = new Course("CIT597", "Dr. Dave");
```

Cont...

- JavaScript has many built-in objects that you can use to perform different activities.
  - Every time you load a page into a web browser, the JavaScript interpreter creates certain objects based on how the HTML is written.
    - This minimal object set is comprised of the **navigator**, **window**, **document**, **location**, and **history** objects.
- Depending on what's contained in the page, there can be other Objects that are also generated by the interpreter.
  - All of these Objects exist in an **Object hierarchy** that can be accessed by calling on the Object **with dot notation.**
- The following chart shows the structure.


Object
RegExp
Function
String
Event
Math
Array
Date
Boolean
screen
window
frameset
navigator
plugin
mimeType
statusbar
menubar
toolbar
document
location
locationbar
personalbar
scrollbars
frame
history
embed
area
applet
tags
classes
link
image
form
layer
ids
document
text
radio
select
textarea
fileUpload
button
checkbox
submit
password
reset
hidden
option

Object Properties and Methods

- A property of an object is basically a predefined variable that you can assign a value to with simple dot notation. Syntax like this:

  objectName.propertyName = value;

  - For instance, if you want to change the background color of the **document Object to blue,** you would access the **bgColor** Property and assign the String "blue" to it like this:

  document.bgColor = "blue";

- A **method of an object** is a predefined function that is assigned to an Object by a browser.

- You invoke a method on an object with the same dot notation like this:

  objectName.methodName();

Date object

Date Object

- Most of the **date and time work** is done with the **Date object**.
- The basic syntax for generating a new date object is as follows:

  var dateObjectName = new Date([parameters])

- The parameter can be:

  new Date("Month dd, yyyy hh:mm:ss")

  new Date("Month dd, yyyy")

  new Date(yy,mm,dd,hh,mm,ss)

  new Date(yy,mm,dd)

By default, **JavaScript will use the browser's time zone and display a date as a full text string:**

Method	Value Range	Description
dateObj.getTime()	0-...	returns Milliseconds since 1/1/70 00:00:00 GMT
dateObj.getYear()	70-...	returns Specified year minus 1900 returns four-digit year for 2000+
dateObj.getFullYear()	1970-...	returns four-digit year
dateObj.getMonth()	0-11	returns Month within the year (January = 0)
dateObj.getDate()	1-31	returns Date within the month
dateObj.getDay()	0-6	returns Day of week (Sunday = 0)
dateObj.getHours()	0-23	returns Hour of the day in 24-hour time
dateObj.getMinutes()	0-59	returns Minute of the specified hour
dateObj.getSeconds()	0-59	returns Second within the specified minute
dateObj.setTime(val)	0-...	sets Milliseconds since 1/1/70 00:00:00 GMT
dateObj.setYear(val)	70-...	sets Specified year minus 1900 sets four-digit year for 2000+
dateObj.setMonth(val)	0-11	sets Month within the year (January = 0)
dateObj.setDate(val)	1-31	sets Date within the month
dateObj.setDay(val)	0-6	sets Day of week (Sunday = 0)
dateObj.setHours(val)	0-23	sets Hour of the day in 24-hour time
dateObj.setMinutes(val)	0-59	sets Minute of the specified hour
dateObj.setSeconds(val)	0-59	sets Second within the specified minute

Cont...

- Example: display current date and time

```
var today = new Date();//browser current date and time
var date = today.getDate();
var month = today.getMonth();
var year = today.getFullYear();
document.write("Today's date: "+date+"/"+month+"/"+year);
```

output: Today's date: 26/09/2026

- Example: to set the date to some past time, like a birth date

```
var myBirthday = new Date("September 11, 2001");
result = myBirthday.getDay();   // result = 2, a Tuesday
myBirthday.setYear(2002);       // bump up to next year
result = myBirthday.getDay();  // result = 3, a Wednesday
```

String object

String Object

- JavaScript strings are for storing and manipulating text.
  - JavaScript imposes no practical limit on the number of characters that a string can hold.
- You have **two ways to assign a string value to a variable.**
  - The simplest is a **basic assignment** statement:

    var myString = “Hello there.”;
  - You can also **create a string object** using the more formal syntax that involves the new keyword and a constructor function like:

    var stringVar = new String(“characters”);

Method	Description
charAt(index)	Returns the character at the specified index.
charCodeAt(index)	Returns a number indicating the Unicode value of the character at the given index.
concat(string)	Combines the text of two strings and returns a new string.
indexOf(string, [start])	Returns the index within the calling String object of the first occurrence of the specified value, or -1 if not found.
lastIndexOf(string,[start])	Returns the index within the calling String object of the last occurrence of the specified value, or -1 if not found.
localeCompare(string2)	Returns a number indicating whether a reference string comes before or after or is the same as the given string in sort order.
Length	Returns the length of the string.
match(regExpression)	Used to match a regular expression against a string.

replace(regExpression,replacer)	Used to find a match between a regular expression and a string, and to replace the matched substring with a new substring.
search(regExpression)	Executes the search for a match between a regular expression and a specified string.
slice(startIndex [,end])	Extracts a section of a string and returns a new string.
split(delimiter [,limit])	Splits a String object into an array of strings by separating the string into substrings.
substr(start [, length])	Returns the characters in a string beginning at the specified location through the specified number of characters.
substring(start, end)	Returns the characters in a string between the two indexes into a string.
toLowerCase()	Returns the calling string value converted to lower case.
toUpperCase()	Returns the calling string value converted to uppercase.
toString()	Returns a string representing the specified object.

Cont…

- Example:

```
var name = new String("Konrad Zuse");
name.concat(" - created the first computer");
name.substring(0,10);
name.indexOf("Zuse");
name.replace("a","#");
name.toUpperCase();
```

- Output:

```
Konrad Zuse - created the first computer
Konrad Zuse
7
Konr#d Zuse
KONR#D ZUSE
```

Document object

Document Object

- Contains information on the current document.
- Document object **contains properties and methods** that can be used to **access the page elements**.
- Properties:
  - **title** - Current document title. If no title is defined, title contains "Untitled."
  - **location** - Full URL of the document.
  - **lastModified** - A Date object-compatible string containing the date the document was last modified.
  - **bgColor** - Background color, expressed as a hexadecimal RGB value (for example, #FFFFF for white). Equivalent to the BGCOLOR attribute of the <BODY> tag.
  - **fgColor** - Foreground (text) color, expressed as a hexadecimal RGB value compatible with HTML syntax. Equivalent to the TEXT attribute of the <BODY> tag.
  - **linkColor** - Hyperlink color, expressed as a hexadecimal RGB value compatible with HTML syntax. Equivalent to the LINK attribute of the <BODY> tag.

Cont...

- **alinkColor** - Activated hyperlink color, expressed as a hexadecimal RGB value. Equivalent to the ALINK attribute of the <BODY> tag.
- **vlinkColor** – visited link color, expressed as hexadecimal RGB value
- **forms[]** - Array of form objects in the document, in the order specified in the source. Each form has its own form object.
- **forms.length** - The number of form objects within the document.
- **links[]** - Array objects corresponding to all HREF links in the document, in the order specified in the source.
- **links.length** - The number of HREF links in the document.
- **anchors[]** - Array of all "named" anchors between the <A NAME=""> and </A> tags within the document.
- **anchors.length** - The number of named anchors in the document.
- **images[]** - Image objects that correspond to each image on the page.
- **applets[]** - Java applet objects that correspond to each applet on the page.
- **embeds[]** - Plugins object that represent each plug-in on the page.

Cont...

- Example: changing link colors

  document.linkColor = “red”;

  document.alinkColor = “blue”;

  document.vlinkColor = “green”;

- Methods:
  - **write("string")** - writes string to the current window. string may include HTML tags.
  - getElementById(name)
  - getElementsByTagName (name)
  - getElementsByClassName (name)
  - **clear( )** - Clears the window.
  - **close( )** - Closes the window.
  - createElement(element)

Reading Assignment
History Object
Number Object

Form Processing and Validation

- It is possible to access forms and form elements from JavaScript.
  - You can set attributes for name, target, action, method, and enctype for a form.
    - Each of these is a property of a FORM object, accessed by all-lowercase versions of those words, as in:

```
document.forms[0].action     //forms array of the document
document.formName.action    //references exact form by name
```

- To change any of these properties, simply assign new values to them:

```
document.registration.action = "login.php"
```

Cont...

- Forms support different events:
  - **onFocus** – an event is triggered with a form object gets input focus.
  - **onBlur** – this event is triggered when a form object loses input focus.
  - **onChange** – an event is triggered when a new item is selected in a list box.
    - This event is also trigged with a text or text area box loses focus and the contents of the box has changed.
  - **onSelect** – an event is triggered when text in a text or text area box is selected.
  - **onSubmit** – an event is triggered when the form is submitted to the server.

Accessing Form Elements

- You **can read data from or set data to** form elements like text field, text area, check box, etc. in many different ways:
  - using form name & element name
  - using forms array & element name
  - using id of the element
- The syntax is as follows:

  value = document.forms[0].filedName.value; //read content

  document.*forms[0].fieldName*.value = *newValue*; //set content
- You can use form name to access form elements using the following syntax:

  value = **document.*formName.filedName*.value;** //read content

  document.*formName.fieldName*.value = *value*; //set content
- It is also possible to use forms array of document object to access form elements.

Cont...

- It is also possible to use **getElementById() method** of JavaScript to access form elements.
- The **getElementById()** method is used to retrieve an element by **using its ID.**
  - The ID is a unique identifier that is assigned to an element in the HTML code.
- The getElementById() method takes the ID as a parameter and returns the element object.
- The syntax to access the form elements is:

```
value = document.getElementById("form element id").value;  //to read
document.getElementById("form element id").value = value;  //to set
```

```
function add() {
   var op1 = document.test.fnum.value;    //using form name directly
   var op2 = document.forms[0].snum.value; //using forms array
   var sum = op1 + op2;
   document.getElementById("result").value = sum;  //using id of the element
 }

<FORM NAME="test" ACTION="abc.php" METHOD="get">
   First number: <INPUT TYPE="text" NAME="fnum" ID="fnum"><br>
   Second number: <INPUT TYPE="text" NAME="snum" ID="snum"><br>
   Result: <INPUT TYPE="text" NAME="result" ID="result"><br>
   <INPUT TYPE="button" NAME="adder" Value="Add" onClick="add()">
```

Cont...

Text boxes and Text areas

- Text fields have a value property.
  - The ***value* property** of the input text box, is both ***readable*** and ***writable***.
- To access text input fields, you can use the syntax:

```
value = document.formName.textName.value; //read content
document.formName.textName.value = value; //set content
```

- **Text box and textarea properties:**
  - **name** - String value of the NAME attribute.
  - **value** - String value of the contents of the field.
  - **defaultValue** - String value of the initial contents of the field.
  - **type** - Specifies what type of object this form field is (e.g. “text” or “Textarea”).

```
<HTML><HEAD><TITLE>Test Input</TITLE>
<SCRIPT LANGUAGE="JavaScript">
function hello ()  {
   var urname = document.test.name.value;
   var gender;
   if(document.test.gender[0].checked)
            gender = document.test.gender[0].value;
   else if(document.test.gender[1].checked)
            gender = document.test.gender[1].value;
   alert ("Hello, " + urname + "! Your are "+ gender);
}
</SCRIPT></HEAD> <BODY>
<FORM NAME="test" ACTION="" METHOD="GET">
Enter your name: <INPUT TYPE="text" NAME="name"><br>
Gender: <INPUT TYPE="radio" NAME="gender" Value="Male">Male<BR>
<INPUT TYPE="radio" NAME=" gender " Value="Female">Female<BR>
<INPUT TYPE="button" NAME="button" Value="Say Hello" onClick="hello()">
</FORM></BODY></HTML>
```

Cont...

- With the help of JavaScript, the form input can easily be checked before sending it over the Internet.
  - It is sent only if the input is valid.
- Form data that is typically checked by JavaScript could be:
  - has the user left required fields empty?
  - has the user entered a valid e-mail address?
  - has the user entered a valid date?
  - has the user entered text in a numeric field?

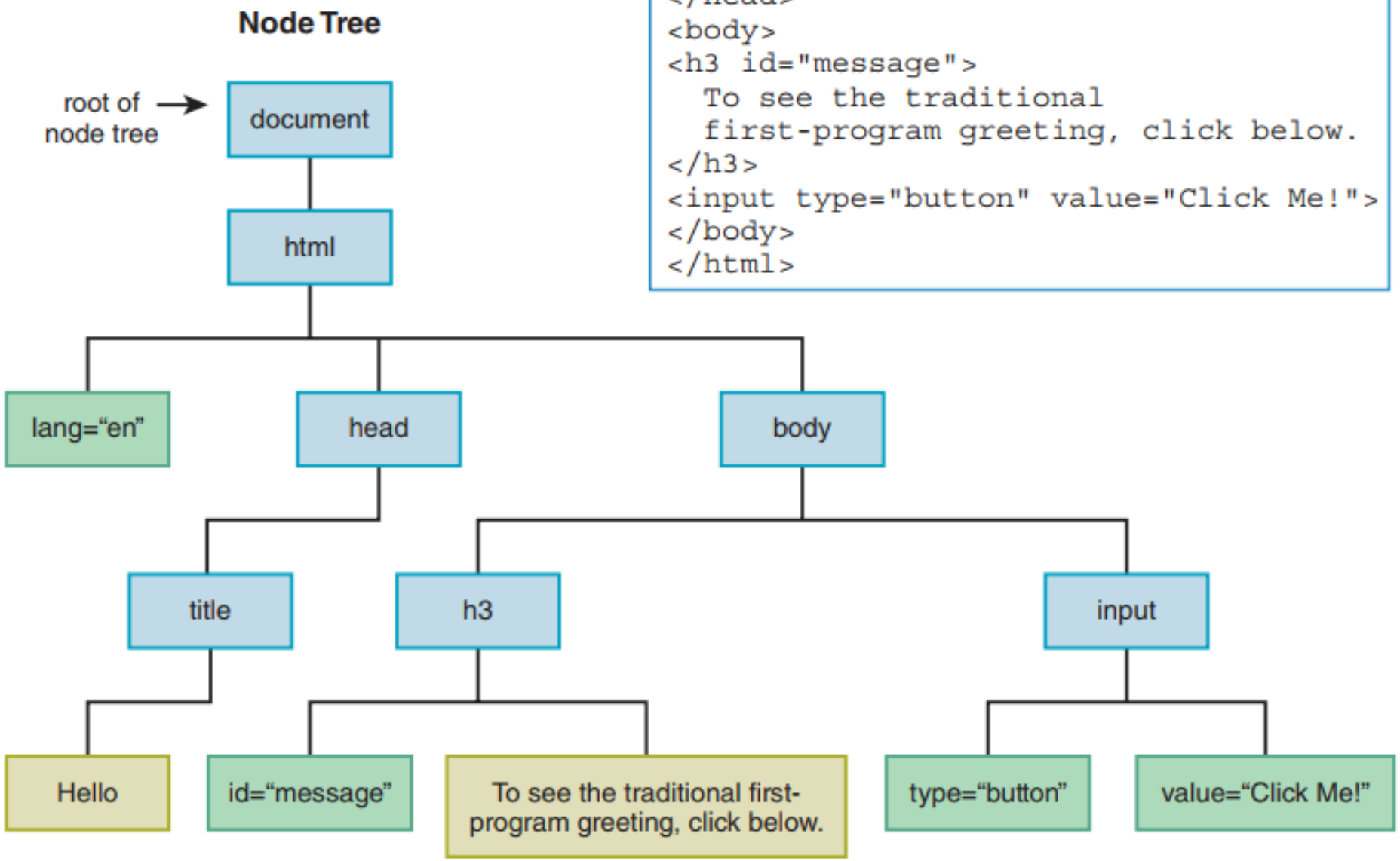
```
<html> <head>
<script language="JavaScript">
function check(form) {
if (document.first.urname.value == "")
   alert("Please enter a string as your name!")
if(fdocument.first.age.value < 0 || form.age.value=="")
   alert("Age should be number and greater than 0");
if (document.first.email.value == "" || document.first.email.value.indexOf('@', 0) == -1)
    alert("No valid e-mail address!");
if(document.first.urmessage.value=="")
    alert("No message written");
}
</script></head> <body>
<h2>  <u> Form validation </u> </h2>
<form name="first">
Enter your name: <input type="text" name="urname"> <br>
Enter your age: <input type="text" name="age"> <br>
Enter your e-mail address: <input type="text" name="email"> <br>
write message: <textarea name="urmessage" cols=40 rows=10></textarea><br><br>
<input type="button" name="validate" value="Check Input" onClick="check(this.form)">
</body></html>
```

HTML DOM

- The Document Object Model, which is normally referred to as the DOM, models all of the parts of a web page document as nodes in a node tree.
  - Each node represents either (1) an element, (2) a text item that appears between an element's start and end tags, or (3) an attribute within one of the elements.
- Every element on an HTML page is accessible in JavaScript through the **DOM**: **Document Object Model**
  - Can modify, add, and remove nodes on the DOM, which will modify, add, or remove the corresponding element on the page.
- When a web page is loaded, the browser creates a **Document Object Model of the page.**

blue: element nodes
yellow: text nodes
green: attribute nodes

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>Hello</title>
</head>
<body>
<h3 id="message">
  To see the traditional
  first-program greeting, click below.
</h3>
<input type="button" value="Click Me!">
</body>
</html>
```



HTML DOM

- With the object model, JavaScript gets all the power it needs to create dynamic HTML:
  - It can change all the HTML **elements** on the page
  - It can change all the HTML **attributes** on the page
  - It can change all the CSS **styles** on the page
  - It can **remove** existing HTML elements and attributes
  - It can **add new** HTML elements and attributes
  - It can **react** to all existing HTML **events** on the page
  - It can **create** new HTML **events** on the page

HTML DOM

- The document object represents your web page. If you want to access any element in an HTML page, you always start with accessing the document object.
- Then you can do a lot of things with the document object:

Action	Example
Finding HTML Elements	document.querySelector(CSS selector);
Adding and Deleting Elements	document.createElement(element);
Changing HTML Elements	element.innerHTML = new html content;
Adding Events Handlers	element.addEventListener('event', handler);

HTML DOM

Finding HTML Elements

- If you want to find the first HTML elements that **match a specified CSS selector** (id, class names, types, attributes, values of attributes, etc), use **the querySelector() method**.

For example, this JavaScript statement will return the first paragraph element of class main:

```
document.querySelector("p.main");
document.querySelectorAll("p.main");
```

```
<body>
        <p>my first paragraph</p>
        <p class="main">my first main paragraph</p>
        <p class="main">my second main paragraph</p>
        <a href="http://www.google.com">google</a>
</body>
```

querySelectorAll() method will return a list of all HTML elements **that match** the specified **CSS query**.

- const param = document.**querySelectorAll**("p.main");
  - param[0], param[1]

HTML DOM

❖ Changing HTML Elements

- The HTML DOM allows JavaScript **to change the content of HTML elements**
- The easiest way to modify the content of an HTML element is by using the **innerHTML property.**

❖ To change the content of an HTML element, use this syntax:

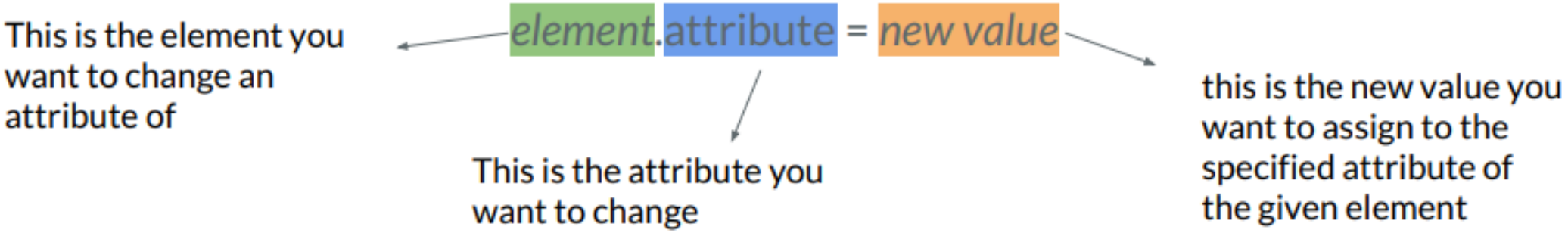


- For example this JavaScript code changes the text inside the h1 element:

```
let header = document.querySelector("h1");
header.innerHTML = "My new heading";
```

HTML DOM

❖ You can also change the value of an HTML attribute.



For example, this javascript code changes the href attribute of an <a> element with id myLink:

```
let myLink = document.querySelector("#myLink");
myLink.href = "http://www.newwebsite.com";
```

```
<a href="http://www.oldwebsite.com" id="myLink">A link to my website</a>
```



```
<a href="http://www.newwebsite.com" id="myLink">A link to my website</a>
```

❖ Changing CSS properties

- To change the style of an HTML element, use this syntax:



For example, this JavaScript code changes the font size of the second <p> element with class par to twice its default value:

```
let pars = document.querySelectorAll("p.par");
pars[1].style.fontSize = "2em";
```

```
p.par {
  font-size: 2em;
}
```



```
pars[1].style.fontSize = "2em";
```

❖ Adding HTML Elements

- To add a new element to the HTML DOM, you must create the element (element node) first, and then append it to an existing element



HTML DOM

❖

This is a paragraph.

This is another paragraph.

```
let para = document.createElement("p");
let node = document.createTextNode("This is new.");
para.appendChild(node);
let element = document.querySelector("#div1");
element.appendChild(para);
```

```
<div id="div1">
<p id="p1">This is a paragraph.</p>
<p id="p2">This is another paragraph.</p>
<p>This is new.</p>
</div>
```

❖ Removing Existing HTML Elements

- To remove an HTML element, you must know the parent of the element
- Then you can use this syntax to remove the element you want:

```
parentElement.removeChild(childElement)
```

This is the parent element you want to remove one of its children elements

This is the child element you want to remove

HTML DOM

❖

This is a paragraph.

This is another paragraph.

```
let parent = document.querySelector("#div1");
let child = document.querySelector("#p1");
parent.removeChild(child);
```

```
<div id="div1">
<p id="p1">This is a paragraph.</p>
<p id="p2">This is another paragraph.</p>
</div>
```